

TUGAS 10 – SELF LEARNING PERFORMANCE TESTING

Pendahuluan

Performance Testing merupakan salah satu bagian penting dalam Software Quality Assurance (SWQA). Pengujian ini dilakukan untuk mengetahui bagaimana suatu aplikasi atau sistem bekerja ketika menerima beban tertentu. Pengujian tidak hanya melihat apakah fungsi aplikasi berjalan dengan benar, tetapi juga memperhatikan kecepatan, kestabilan, kapasitas, dan penggunaan resource.

Aplikasi yang memiliki fitur lengkap tetapi membutuhkan waktu lama untuk memberikan response atau mengalami error ketika banyak pengguna mengaksesnya tetap dapat dikatakan memiliki masalah kualitas. Karena itu, Performance Testing diperlukan untuk menemukan masalah performa lebih awal dan memastikan sistem mampu memenuhi kebutuhan pengguna.

1. Lighthouse Report

1.1 Pengertian Lighthouse

Lighthouse merupakan tools open-source yang digunakan untuk melakukan audit terhadap halaman website. Lighthouse dapat dijalankan melalui Chrome DevTools maupun melalui command line. Tools ini membantu developer dan tester mengevaluasi kualitas sebuah website dari beberapa aspek.

- Performance, untuk mengukur performa dan kecepatan halaman.
- Accessibility, untuk mengevaluasi kemudahan website digunakan oleh berbagai pengguna.
- Best Practices, untuk memeriksa penerapan praktik pengembangan web yang baik.
- SEO, untuk mengevaluasi beberapa aspek yang mendukung optimasi mesin pencari.
- Progressive Web App (PWA), untuk memeriksa kemampuan website dalam memenuhi karakteristik PWA.

1.2 Lighthouse Performance Report

Setelah audit selesai, Lighthouse menghasilkan report yang berisi score, metric, dan rekomendasi perbaikan. Bagian Performance menjadi perhatian utama ketika Lighthouse digunakan untuk mengevaluasi kecepatan website.

1.3 Metric Lighthouse

a. First Contentful Paint (FCP)

FCP menunjukkan waktu yang dibutuhkan browser sampai menampilkan konten pertama kepada pengguna. Contohnya berupa teks, gambar, atau elemen lain yang pertama kali muncul pada halaman.

b. Largest Contentful Paint (LCP)

LCP mengukur waktu sampai elemen konten terbesar yang terlihat pada viewport berhasil ditampilkan. Metric ini membantu mengetahui seberapa cepat konten utama halaman dapat dilihat pengguna.

c. Total Blocking Time (TBT)

TBT mengukur total waktu ketika halaman mengalami blocking sehingga proses interaksi pengguna dapat terganggu. Nilai yang lebih rendah menunjukkan halaman lebih responsif.

d. Cumulative Layout Shift (CLS)

CLS mengukur perubahan posisi elemen halaman yang terjadi secara tidak terduga selama proses loading. Layout yang stabil membuat website lebih nyaman digunakan.

e. Performance Score

Lighthouse memberikan score yang menggambarkan kondisi performa halaman berdasarkan hasil audit. Score dapat digunakan sebagai indikator awal untuk mengetahui apakah halaman membutuhkan optimasi.

1.4 Manfaat Lighthouse

- Mengevaluasi performa halaman website.
- Menemukan masalah yang menyebabkan website lambat.
- Mengetahui metric performa secara terukur.
- Memberikan rekomendasi untuk optimasi.
- Membantu meningkatkan pengalaman pengguna.

2. Performance Report

2.1 Pengertian Performance Report

Performance Report adalah laporan yang berisi hasil pengujian performa sebuah aplikasi atau sistem. Report digunakan untuk mendokumentasikan kondisi sistem selama pengujian dan membandingkan hasilnya dengan performance requirement atau target yang telah ditentukan.

Performance Report yang baik tidak hanya berisi angka, tetapi juga menjelaskan kondisi pengujian, skenario, hasil, masalah yang ditemukan, serta rekomendasi perbaikan.

2.2 Informasi dalam Performance Report

- Test environment, yaitu lingkungan tempat pengujian dilakukan.
- Test scenario, yaitu skenario atau aktivitas yang diuji.
- Jumlah virtual user atau concurrent user.
- Durasi pengujian dan pola beban.
- Response time.
- Throughput.
- Error rate.
- Latency.
- Penggunaan CPU, memory, disk, dan network.
- Kesimpulan dan rekomendasi.

2.3 Metric Performance Report

a. Response Time

Response Time adalah waktu yang dibutuhkan sistem untuk memberikan response setelah menerima request. Response time menjadi salah satu indikator utama karena berhubungan langsung dengan seberapa cepat pengguna mendapatkan hasil.

b. Throughput

Throughput menunjukkan jumlah request atau transaksi yang dapat diproses sistem dalam periode tertentu. Contohnya adalah 1.000 request per menit.

c. Error Rate

Error Rate menunjukkan persentase request yang mengalami kegagalan. Misalnya dari 1.000 request terdapat 20 request gagal, maka error rate adalah 2%.

d. Latency

Latency merupakan waktu yang diperlukan dalam proses komunikasi sebelum response diterima. Latency yang tinggi dapat membuat aplikasi terasa lambat.

e. Resource Usage

Resource Usage menunjukkan penggunaan resource seperti CPU, memory, disk, dan network. Penggunaan resource yang terlalu tinggi dapat menjadi indikasi adanya bottleneck.

2.4 Tujuan Performance Report

Mendokumentasikan hasil pengujian.
Mengetahui kondisi performa sistem.
Menemukan masalah atau bottleneck.
Membandingkan hasil dengan target performa.
Menjadi dasar untuk melakukan optimasi.

Mengetahui perubahan performa setelah perbaikan.

3. JMeter & k6

3.1 Apache JMeter

Apache JMeter merupakan tools open-source yang digunakan untuk melakukan Performance Testing, Load Testing, dan pengujian terhadap berbagai service seperti HTTP/HTTPS dan API. JMeter dapat mensimulasikan banyak virtual user yang mengakses sistem secara bersamaan.

3.2 Komponen Dasar JMeter

- Test Plan: struktur utama yang berisi konfigurasi dan skenario pengujian.
- Thread Group: menentukan jumlah virtual user, ramp-up time, dan jumlah pengulangan.
- Sampler: digunakan untuk mengirim request ke server, misalnya HTTP Request.
- Listener: digunakan untuk melihat dan menganalisis hasil pengujian.

3.3 Contoh Skenario JMeter

Contohnya sebuah website memiliki halaman login dan ingin diuji dengan 100 user secara bersamaan. Tester dapat membuat Thread Group dengan 100 virtual user, menentukan ramp-up time, lalu menambahkan HTTP Request untuk proses login. Setelah test selesai, hasil dapat dianalisis melalui response time, throughput, dan error rate.

3.4 k6

k6 merupakan tools Performance Testing yang menggunakan JavaScript untuk membuat test script. k6 banyak digunakan untuk pengujian API, automation, dan workflow CI/CD karena test dapat ditulis sebagai kode dan dijalankan melalui command line.

3.5 Contoh Script k6

```
import http from 'k6/http';
import { check } from 'k6';

export const options = {
  vus: 10,
  duration: '30s',
};

export default function () {
  const response = http.get('https://example.com');

  check(response, {
    'status code is 200': (r) => r.status === 200,
```

```
});  
}
```

3.6 Penjelasan Script

- `vus`: jumlah virtual user yang digunakan.
- `duration`: lama pengujian.
- `http.get()`: mengirim request GET.
- `check()`: melakukan validasi terhadap response.
- `status code 200`: menunjukkan request berhasil pada skenario sederhana tersebut.

3.7 Perbandingan JMeter dan k6

Aspek	JMeter	k6
Interface	GUI	Script/CLI
Pendekatan	Visual	Code-based
Bahasa	Java-based tool	JavaScript
Virtual User	Thread Group	VUs
Automation	Bisa	Sangat cocok
CI/CD	Bisa	Sangat cocok

JMeter cocok digunakan ketika tester membutuhkan interface visual untuk membuat skenario. k6 lebih cocok untuk pengguna yang terbiasa dengan scripting dan membutuhkan automation serta integrasi dengan pipeline development.

4. Rekap Performance Testing Berdasarkan Video

Berdasarkan materi video Performance Testing pada sesi pembelajaran, Performance Testing merupakan proses untuk mengetahui bagaimana performa sebuah aplikasi atau sistem ketika diberikan beban tertentu. Fokusnya bukan hanya pada keberhasilan fungsi, tetapi juga pada kecepatan, kestabilan, kemampuan menangani beban, dan penggunaan resource sistem.

4.1 Tujuan Performance Testing

1. Mengetahui kecepatan response aplikasi.
2. Mengetahui kemampuan sistem dalam menangani beban.
3. Mengetahui kestabilan aplikasi.
4. Menemukan bottleneck.
5. Mengetahui batas kemampuan sistem.
6. Memantau penggunaan resource.
7. Mengetahui kemampuan sistem ketika jumlah pengguna meningkat.
8. Memastikan sistem memenuhi kriteria performa.

4.2 Metric Performance Testing

Materi video membahas beberapa metric yang digunakan untuk menilai performa sistem. Metric tersebut memberikan gambaran mengenai kecepatan, kapasitas, error, dan penggunaan resource.

- Response Time: waktu yang dibutuhkan sistem untuk memberikan response.
- Throughput: jumlah request atau transaksi yang diproses dalam periode tertentu.
- Error Rate: persentase request yang mengalami kegagalan.
- Latency: waktu yang dibutuhkan dalam proses komunikasi.
- Resource Usage: penggunaan CPU, memory, disk, network, dan resource OS lainnya.

4.3 Jenis Performance Testing

a. Load Testing

Load Testing digunakan untuk mengetahui performa sistem ketika diberikan beban sesuai kondisi penggunaan yang diharapkan. Contohnya adalah simulasi sejumlah user yang mengakses aplikasi secara bersamaan.

b. Stress Testing

Stress Testing dilakukan dengan memberikan beban yang lebih tinggi dari kondisi normal. Tujuannya untuk mengetahui batas kemampuan sistem dan bagaimana sistem berperilaku ketika menerima beban berlebih.

c. Spike Testing

Spike Testing digunakan untuk menguji sistem ketika terjadi perubahan beban secara tiba-tiba, misalnya jumlah pengguna meningkat drastis dalam waktu singkat.

d. Volume Testing

Volume Testing digunakan untuk mengetahui kemampuan sistem ketika menangani data dalam jumlah besar. Pengujian ini dapat membantu menemukan masalah pada database atau komponen yang berhubungan dengan pengolahan data.

e. Endurance/Soak Testing

Endurance atau Soak Testing dilakukan dengan memberikan beban dalam waktu yang lama. Tujuannya untuk mengetahui kestabilan sistem dan menemukan masalah yang muncul setelah sistem bekerja dalam waktu panjang, misalnya penurunan performa atau memory leak.

f. Scalability Testing

Scalability Testing digunakan untuk mengetahui kemampuan sistem dalam menghadapi peningkatan jumlah pengguna atau beban. Pengujian ini membantu menentukan apakah sistem masih dapat berkembang ketika kebutuhan meningkat.

4.4 Tools Performance Testing

Beberapa tools yang dibahas dalam materi Performance Testing antara lain:

- Apache JMeter, digunakan untuk melakukan load dan performance testing.
- k6, digunakan untuk performance testing berbasis scripting.
- HP LoadRunner, digunakan untuk melakukan performance dan load testing.

4.5 Best Practice Performance Testing

- Menggunakan environment yang menyerupai production agar hasil testing lebih relevan.
- Melakukan testing sejak awal dan secara berkala, bukan hanya ketika aplikasi selesai.
- Menggunakan test scenario yang realistis berdasarkan aktivitas pengguna.
- Membuat baseline sebagai acuan performa dan membandingkan hasil pengujian berikutnya dengan baseline tersebut.

4.6 Common Performance Issues

- Loading time terlalu lama.
- Bottleneck pada aplikasi atau infrastruktur.
- Response time yang buruk.
- Masalah scalability ketika jumlah user meningkat.
- Penggunaan CPU, memory, disk, atau network terlalu tinggi.

4.7 Tahapan Performance Testing

1. Mengidentifikasi environment yang akan digunakan.
2. Menentukan performance criteria atau target yang ingin dicapai.
3. Menentukan test scenario berdasarkan aktivitas pengguna.
4. Menentukan workload seperti jumlah user, request, dan durasi.
5. Menjalankan pengujian menggunakan tools yang sesuai.
6. Mengumpulkan metric seperti response time, throughput, error rate, dan resource usage.
7. Menganalisis hasil dan membandingkannya dengan target.
8. Melakukan perbaikan terhadap masalah yang ditemukan.
9. Melakukan retesting untuk memastikan perbaikan berhasil.

4.8 Contoh Test Case

No	Test Case	Load	Durasi	Expected Result
1	Login	100 users	5 menit	Response stabil
2	Open Dashboard	100 users	5 menit	Loading tetap baik
3	Search Data	200 users	5 menit	Tidak terjadi error
4	API Request	500 users	10 menit	Error rate rendah

4.9 AI dalam Performance Testing

Materi juga membahas perkembangan penggunaan AI dalam Performance Testing. AI dapat membantu proses analisis dan meningkatkan efisiensi pengujian.

- Anomaly detection untuk membantu menemukan pola performa yang tidak normal.
- Root cause analysis untuk membantu mencari penyebab masalah.
- Optimasi test scenario dan proses pengujian.
- Membantu membuat atau memperbaiki test script.
- Menganalisis hasil testing dalam jumlah besar.
- Mendukung proses testing yang lebih otomatis dan adaptif.

Pemanfaatan AI dapat membantu tester menghemat waktu dalam analisis, tetapi hasilnya tetap perlu divalidasi agar keputusan testing tetap akurat.

Kesimpulan

Performance Testing merupakan bagian penting dalam Software Quality Assurance untuk memastikan aplikasi tidak hanya berfungsi dengan benar, tetapi juga cepat, stabil, dan mampu menangani beban pengguna. Lighthouse dapat digunakan untuk melakukan audit performa website, Performance Report digunakan untuk mendokumentasikan hasil pengujian, sedangkan JMeter dan k6 dapat digunakan untuk melakukan simulasi beban.

Berdasarkan materi video, Performance Testing mencakup berbagai jenis pengujian seperti Load Testing, Stress Testing, Spike Testing, Volume Testing, Endurance/Soak Testing, dan Scalability Testing. Metric seperti Response Time, Throughput, Error Rate, Latency, dan Resource Usage digunakan untuk menganalisis hasil. Dengan environment yang sesuai, skenario realistis, baseline, serta proses testing dan retesting yang terstruktur, masalah performa dapat ditemukan lebih awal dan kualitas aplikasi dapat ditingkatkan.